

Structure and Interpretation of Computer Programs (SICP)

Harold Abelson

Gerald Jay Sussman

Julie Sussman

Anotações do cap. The Elements of Programming, 1-7

Introdução

Processos computacionais podem ser definidos como entidades abstratas presentes nos computadores. À medida que esses processos se desenrolam, eles manipulam outro conjunto de coisas abstratas chamadas de *dados (data)*. A evolução e execução desses processos é dirigida por um conjunto e padrões de regras chamados de *programas*. Programas são criados justamente para dirigir processos.

Todos esses conceitos são de fato abstratos, e não são materiais, mas mesmo assim são reais. Eles manipulam dados, performam operações matemáticas, e guiam tudo aquilo que conhecemos que necessita de algum tipo de intervenção computacional. Por sua vez, programas são construídos através de expressões simbólicas, chamadas de *linguagens de programação*, que prescrevem as tarefas que queremos que os nossos processos executem.

Processos computacionais executam de maneira extremamente precisa em computadores que estão funcionando da maneira ideal, mas programadores (aqueles que escrevem as instruções que os processos devem executar) devem ser capazes de aprender a entender e antecipar as consequências daquilo que escrevem, pois uma instrução mal escrita e direcionada pode gerar problemas (chamados de *bugs* ou *glitches*), e mesmo que pequenos, podem ocasionar consequências complexas e até mesmo severas.

A programação no mundo real, no entanto, exige cuidado, competência e sabedoria. Um pequeno erro em um programa de projeto assistido por computador, por exemplo, pode levar ao colapso catastrófico de um avião ou de uma barragem, ou à autodestruição de um robô industrial.

Programadores experientes têm a habilidade de organizar os seus programas de uma maneira que eles conseguem ter uma segurança confiável de que os processos resultantes irão executar as tarefas pretendidas e instruídas anteriormente. Eles conseguem visualizar o comportamento dos seus sistemas à medida em que eles avançam, e sabem como estruturar programas de modo que quando problemas que não podem ser antecipados apareçam, eles não causem consequências catastróficas corrompendo um trabalho importante, e além disso, de uma maneira que eles possam *debugar* quando acontecer. Sistemas bem desenhados são feitos de maneira modular, pois isso permite que cada parte seja construída, trocada e debugada separadamente, sem que isso afete todo o funcionamento do sistema.

Lisp

Lisp é uma família de linguagens de programação criada por John McCarthy em 1958, originalmente desenvolvida para o processamento de listas e a manipulação de símbolos. Sua principal característica é a capacidade de representar o código como estruturas de dados, permitindo que programas manipulem e transformem outras expressões de código. Nesse livro *Structure and Interpretation of Computer Programs* (SICP), é utilizado o dialeto Scheme, uma variante minimalista de Lisp, por sua sintaxe simples e por facilitar a exploração de conceitos fundamentais da programação, como abstração, recursão, funções, estruturas de dados e avaliação de expressões.

Embora Lisp não seja uma linguagem de uso predominante no mercado, suas características únicas fazem dela um excelente meio para estudar conceitos importantes de programação e relacioná-los aos recursos da linguagem que os sustentam. Uma das características mais significativas é que os procedimentos, isto é, as descrições de processos em Lisp, podem ser representados e manipulados como dados da própria linguagem. Isso permite explorar técnicas poderosas de projeto de programas que ultrapassam a distinção tradicional entre dados “passivos” e processos “ativos”. Essa flexibilidade também torna Lisp especialmente adequada para o desenvolvimento de programas que manipulam outros programas como dados, como interpretadores e compiladores.